

Lean Design of Optimized Level-1 BLAS

LAK Working Note #2

deval-d@stanford.edu

13 May 2026

Abstract

The level-1 BLAS consists of $O(n)$ vector-vector routines with low arithmetic intensity. Kernels are divided into two cases: routines without loop-carried dependencies and routines with loop-carried dependencies. Direct scalar loops are sufficient for the first case; the second case requires independent accumulators with explicit manual SIMD to avoid serializing the reduction. With contiguous vector views in **lak**, this gives a lean implementation in which most kernels remain ordinary loops and only the dependency-chain kernels contain manual SIMD instructions.

The compiler discussed is **rustc 1.94.0-nightly**. Other compilers may not have similar behavior.

1 Dependency Chains

The level-1 BLAS routines operate on vectors. Examples include scaling a vector, adding one vector to another, copying data, taking a dot product, and computing vector norms. These routines all have $O(n)$ work, but they do not all present the same optimization problem to the compiler. The important distinction in this note is whether the natural scalar loop has a *loop-carried dependence*. A loop-carried dependence occurs when one iteration of the loop needs a value produced by an earlier iteration. When no such dependence exists, the iterations can be treated as independent pieces of work. When such a dependence does exist, the loop looks sequential unless the implementation rewrites it to expose more parallelism.

The **AXPY** routine is the simple case. It computes

$$y_i \leftarrow \alpha x_i + y_i. \quad (1)$$

Each index i updates a different element of y . The calculation of y_7 does not depend on the result of the computation of y_6 . This is the elementwise structure shown on the left of Figure 1. Once the compiler sees a clean loop over contiguous slices, the elementwise structure is explicit. On a modern target, LLVM can often turn that scalar loop into SIMD instructions **automatically**, even without the source code mentioning SIMD directly. This will be covered more in sections 2 and 3.

The dot product has a different shape. It computes

$$x \cdot y = \sum_{i=0}^{n-1} x_i y_i. \quad (2)$$

A direct scalar implementation naturally introduces a running sum:

```
pub fn dot<T>(
    x: VecRef<'_, T>,
    y: VecRef<'_, T>,
) -> T
where
    T: Default
        + Copy
        + Fma
        + Mul<Output=T>
        + Add<Output=T>,
{
    // ensures buffers have equal length
    assert_ne!(x, y);

    // access stored @[T] buffer;
    let xdata = x.as_slice();
    let ydata = y.as_slice();

    // running sum
    let mut sum = T::default();
    for (&xi, &yi) in xdata.iter().zip(ydata.iter()) {
        // dependency chain
        // sum += xi * yi
        sum = xi.fma(yi, sum);
    }

    sum
}
```

The inner loop does not merely compute an independent output for each index. Instead, each iteration reads the current value of **sum** and produces the next value of **sum**. The value produced at iteration i is therefore an input to iteration $i + 1$. This is the dependency chain shown on the right side of Figure 1. This does not mean the dot product is impossible to vectorize. The products $x_i y_i$ are independent. What is serial in the naive loop is the single accumulated **sum** state. To use SIMD well, the implementation *must* change the *shape* of the computation: instead of one running sum, it uses several independent *partial* sums and combines them at the end, outside of the inner loop. This rewrite is the main subject of section 3.

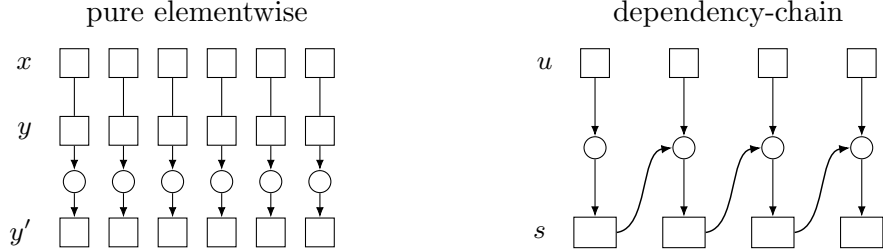


Figure 1: Classification of the level-1 kernels. Elementwise routines expose independent work directly. Dependency-chain routines carry state between iterations, hiding the available parallelism behind a scalar recurrence.

This gives the organizing principle for the rest of the note. For elementwise kernels, the main job is to expose a clean contiguous loop and let the compiler automatically emit SIMD. Even if the compiler-generated SIMD code is minimally unrolled, the very low arithmetic intensity of the level-1 BLAS allows such code to still be roughly optimal. For reduction kernels, the main job is to re-design the implementation to remove scalar recurrence from the hot loop via introducing independent accumulators.

2 AXPY

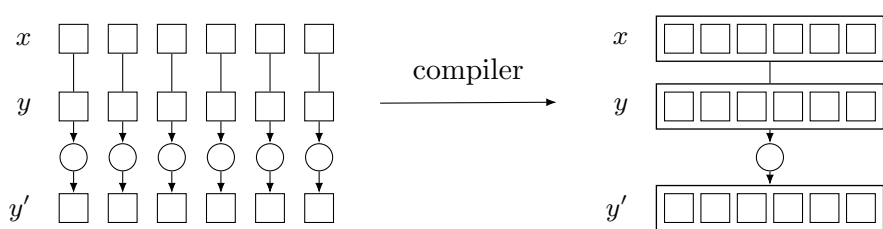


Figure 2: Independent elementwise iterations are automatically widened into a single SIMD operation.

3 DOT

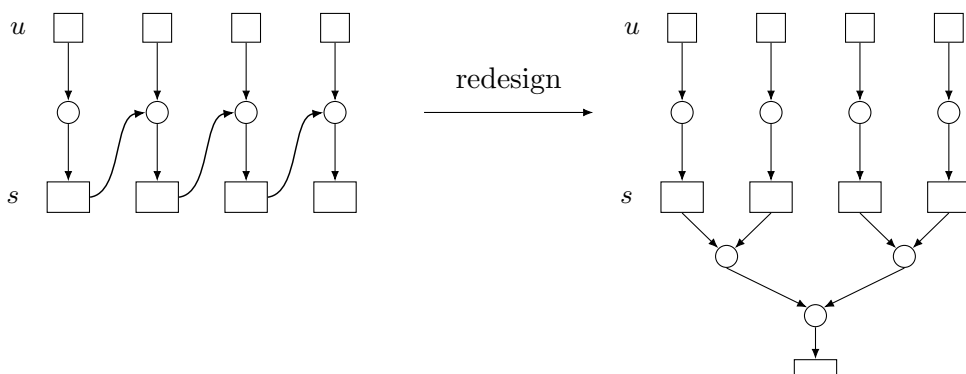


Figure 3: A running accumulator creates a loop-carried dependency. Splitting the work across independent accumulators exposes parallelism inside the loop; the partial sums are reduced afterward.